

When a Straight Line Is Not Enough

How can we choose a model that captures the pattern in the data without simply choosing the most complicated model?

Start from the [challenge landing page](#). You need a Google account for Colab and an AI partner. Save a copy of the starter notebook to Drive, then work in order. The page links to your notebook, dataset and code snippets.

1 · Look at the data

0-8 min

Load the CSV, inspect x and y, and make a scatterplot. Before fitting: Would one straight line describe the relationship well? What would you look for after fitting to decide whether the line misses something?

2 · Establish a baseline

8-18 min

Create the fixed 80/20 train/test split. Reserve the test set. Fit linear regression on training data; inspect training RMSE, R2, the fitted line and residuals. What residual pattern appears? What would you expect if the relationship were captured well?

AI partner interaction 1

Think / Request / Assess

My linear regression residuals show a curved pattern. I still want to use linear regression as the estimator. Is there a way to change how the input features are represented so the model can capture curvature? Do not give me a complete solution. Help me understand the idea and decide what I should try next.

Record and check

What did your AI partner suggest? Why does it make sense based on your residual plot? Check that idea in the next checkpoint.

Compare and check

3 · Experiment with complexity

18-30 min

Try polynomial degrees 1-6 with the supplied support code. Split only the training data into fitting and validation sets. Compare training and validation RMSE. What happens to training error? Does more complexity always help validation? Choose a provisional degree and explain why.

AI partner interaction 2

Predict before asking

I chose a polynomial degree using one training/validation split. If I changed the random split, could my conclusion change? What experiment could I use to make this model comparison more reliable? Explain the reasoning before giving me code.

4 · Test the decision, not the test set

30-43 min

Use five shuffled folds (seed 42) on training data only. Compare degrees 1-6 by mean CV RMSE, standard deviation and individual fold scores. Is your single-split choice still your choice? Which models vary most? Why consider both average performance and variability? Fold SD is spread, not a confidence interval.

Keep the steps together

5 · Keeping the modelling steps together

43-55 min

Think about generating polynomial features and scaling them. Which transformations learn from observed values? Predict when they should be fitted, then ask your AI partner:

AI partner interaction 3

Reasoning before syntax

My model now requires transformations before linear regression, and I am evaluating it using cross-validation. When cross-validation creates each training fold and validation fold, when should those transformations be learned? Is there a scikit-learn tool that can keep the transformations and model together as one modelling workflow? Explain why this matters before showing me how to use it.

After the conversation

Build and evaluate the complete workflow using sklearn Pipeline. Fit transformations only on each training fold, then apply them to its validation fold. Fixed polynomial powers do not learn means or variances; scaling does. Scaling is optional for this unregularized estimator and should not materially improve its scores. Explain why Pipeline keeps fitting and prediction consistent.

Decide and explain

6 • Make the decision

55-60 min

Choose your degree before using the test set. Use mean CV RMSE, fold variation, complexity and plots to justify it. Record your choice and expected performance.

7 • Final evaluation

60-68 min

Fit the chosen workflow on all training data. Evaluate once on the untouched test set: report RMSE, R^2 and a residual plot. Is test performance reasonably consistent with CV? What would concern you? Do not retune after seeing the test score.

8. Report back

68-75 min

Work together to answer the eight challenge questions in your group's Word document. Support your group answers with evidence from the completed notebook, including plots, RMSE values, cross-validation results, fold variability, and the final test evaluation.

1. Which polynomial degree did your group choose?
2. What evidence supports that choice?
3. Why was one training/validation split not enough to make the decision confidently?
4. What did the variation across the cross-validation folds tell you?
5. What problem does Pipeline help solve in this experiment?
6. What did your AI partner help clarify during the challenge?
7. After seeing the final test results, do you still think your model choice was reasonable? Explain using the evidence.
8. What is one thing your group would investigate next if you continued this experiment?

Submission and expectations

1. What to submit

Submit one ZIP file per group to Moodle. The ZIP file should contain:

- The group's completed Google Colab notebook (.ipynb), including code, plots, tables, cross-validation results, and final test evaluation.
- One Word document (.docx) containing the group's answers to the eight challenge questions.

At the top of the Word document include:

Challenge 1: When a Straight Line Is Not Enough

Group members:

Student IDs:

Only one member of the group needs to submit the ZIP file.

Provide one set of group answers; separate answers from every group member are not required.

An AI partner transcript is not required. For the AI partner question, explain in your own words what the AI partner helped your group understand. Do not paste the conversation.

2. Use your AI partner while coding

Use your AI partner during the coding portion of the challenge. Ask it for help with syntax, debugging, understanding scikit-learn tools, or checking a coding approach. If it suggests code, read it, understand what it does, run it, and verify that it performs the experiment you intended. The AI partner can help your group write code, but it should not make the modelling decision for you.

TRACE for code

Think: What do we need this code to do? Request: ask a focused coding question. Assess: inspect the suggestion. Check: run it and verify the outputs. Explain: be able to describe what the code does in your own words.

3. What we are looking for

- Decisions supported by evidence from your plots, RMSE values, CV results, fold variability and final evaluation.
- A notebook that can be followed and connects the code to the intended experiment.
- Group answers that explain your reasoning clearly in your own words.
- Correct test-set discipline: keep it closed until Checkpoint 7 and do not retune after seeing the test result.
- Evidence that your group assessed the AI partner's suggestions rather than simply accepting them.

You will work in different teams in future challenges. The feedback you receive on this challenge will help you apply what you have learned in your next team.